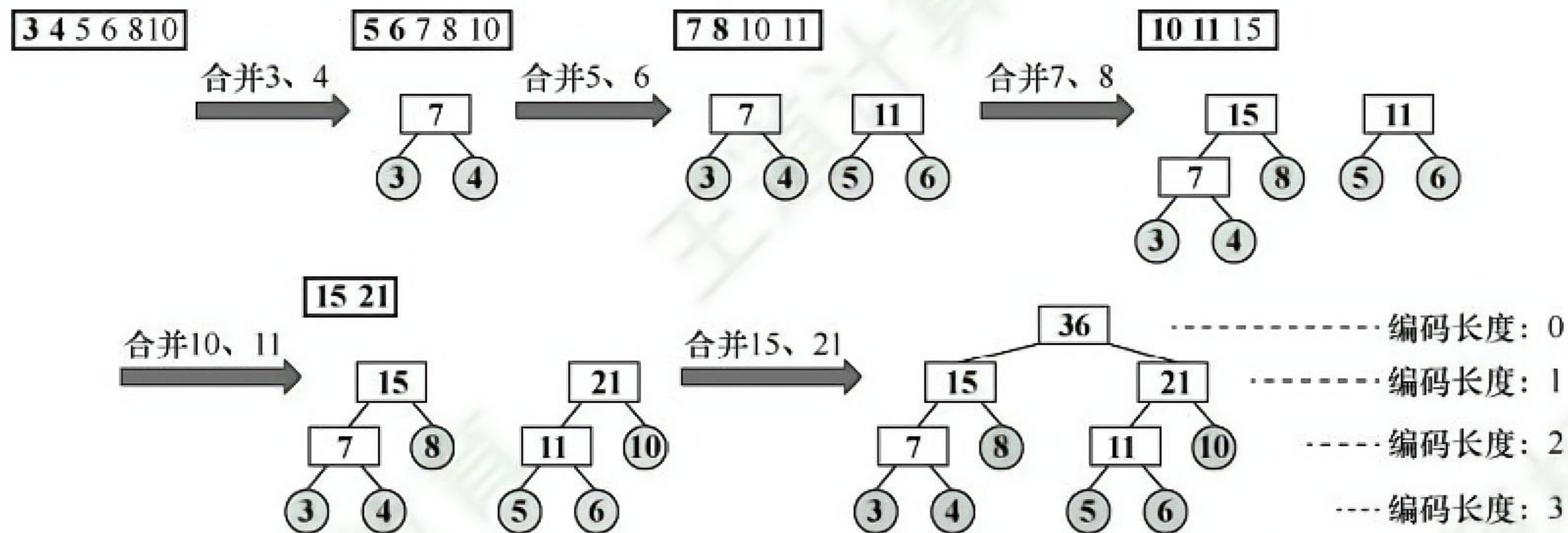




对叶结点的哈夫曼编码, 共有 4 个长度为 3 的叶结点、2 个长度为 2 的叶结点, 编码的加权平均长度为 $[(3+4+5+6) \times 3 + (8+10) \times 2] / (3+4+5+6+8+10) = 2.5$ 。

6.1.3 答案与解析

一、单项选择题

01. A

本题是北京交通大学考研真题，不同教材对路径的定义可能略有不同，顶点之间关联的边也可理解为路径的构成要素。对于 B，路径的定义中并没有要求是不同顶点，比如简单回路的第一个顶点和最后一个顶点是可以相同的，此外 B 也没有说明这些顶点之间有边相联。

02. D

若一个无向图有 n 个顶点和 $n-1$ 条边，可以使它连通但没有环（即生成树），但若再加一条边，在不考虑重边的情形下，则必然会构成环。

03. B

强连通图是有向图，与题意矛盾，A 错误；对无向连通图做一次深度优先搜索，可以访问到该连通图的所有顶点，B 正确；有回路的无向图不一定是连通图，因为回路不一定包含图的所有结点，C 错误；连通图可能是树，也可能存在环，D 错误。

04. C

图与树的区别是逻辑上的区别，而不是边数的区别，图的边数也可能小于树的边数，A 错误；若 E' 中的边对应的顶点不是 V' 的元素， V' 和 $\{E'\}$ 无法构成图，B 错误；无向图的极大连通子图称为连通分量，C 正确；图的遍历要求每个结点只能被访问一次，且若图非连通，则从某一顶点出发无法访问到其他全部顶点，D 的说法不准确。

05. C

强连通有向图的任何顶点到其他所有顶点都有路径，但未必有弧；无向图任意顶点的入度等于出度，但有向图未必满足；若边集中的某条边对应的某个顶点不在对应的顶点集中，则有向图的边集的子集和顶点集的子集无法构成子图。

06. C

考查至少有多少个顶点的情形，我们考虑该非连通图最极端的情况，即它由一个完全图加一个独立的顶点构成，此时若再加一条边，则必然使图变成连通图。在 $28 = n(n-1)/2 = 8 \times 7/2$ 条边的完全无向图中，总共有 8 个顶点，再加上 1 个不连通的顶点，共 9 个顶点。

07. A

对于连通无向图，边最少即构成一棵树的情形；对于强连通有向图，边最少即构成一个有向环的情形。

08. D

因为在具有 n 个顶点、 e 条边的无向图中，有 $\sum_{i=1}^n \text{TD}(v_i) = 2e$ ，所以求得度为 2 的顶点数为 7，从而共有 16 个顶点。

09. D

在有向图中，顶点的度等于入度与出度之和。 n 个顶点的有向图中，任意一个顶点最多还可以与其他 $n-1$ 个顶点有一对指向相反的边相连。注意数据结构中仅讨论简单图。

10. D

5 个顶点构成一个完全无向图，需要 $n(n-1)/2 = 10$ 条边；再加上 1 条边后，能保证第 6 个顶点必然与此完全无向图构成一个连通图，所以共需 11 条边。

11. D

一个连通图的生成树是一个极小连通子图，显然它是无环的，所以选项 II、III 正确。极大连通子图称为连通分量， G' 连通但非连通分量。这里再补充一下“极大连通子图”：若图本来就不是连通的，且每个子部分包含其本身的所有顶点和边，则它就是极大连通子图。

12. C

初始考虑只有 51 个顶点的无向图 G ，此时 G 中每个顶点都是连通分量，问题转化为向 G 中添加 21 条边，如何添加这 21 条边使得连通分量数目最多。若向两个不同的连通分量之间添加边，则连通分量数目会减 1，所以应尽可能地将这 21 条边加入同一个连通分量且让其接近完全图，含有 7 个顶点的完全图有 21 条边，所以用 7 个顶点构成一个含有 21 条边的连通分量，剩下 $51 - 7 = 44$ 个顶点对应 44 个连通分量，共有 45 个连通分量。

13. B

强连通分量是极大强连通子图，任意两个顶点之间有方向相反的两条路径。由定义不难得出，若一个顶点只有出边或入边，则该顶点必定单独构成一个连通分量。图中，顶点 B 只有出

边, 其他所有顶点都不可能到顶点 B 的路径, 所以顶点 B 单独构成一个强连通分量。在顶点 A、C、D、E 中, 任意两个顶点之间都有方向相反的两条路径, 所以可构成一个强连通分量。

14. B

n 个顶点的生成树是具有 $n-1$ 条边的极小连通子图, 因为 n 个顶点构成的环共有 n 条边, 去掉任意一条边就是一棵生成树, 所以共有 n 种情况, 所以可以有 n 棵不同的生成树。

15. C

n 个结点的树有 $n-1$ 条边, 假设森林中有 x 棵树, 将每棵树的根连到一个添加的结点, 则成为一棵树, 结点数是 $n+1$, 边数是 $e+x$, 从而可知 $x=n-e$ 。

另解: 设森林中有 x 棵树, 则再用 $x-1$ 条边就可将所有的树连接成一棵树, 此时边数 $+1 =$ 顶点数, 即 $e+(x-1)+1=n$, 所以 $x=n-e$ 。

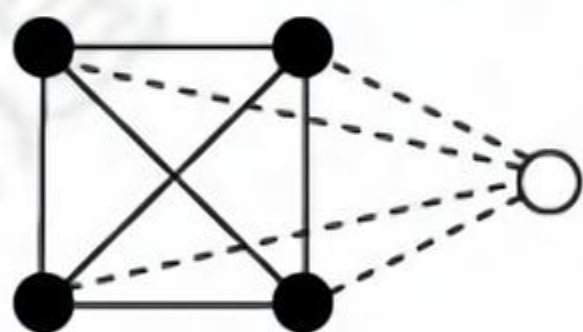
16. A

每条边都连接了两个顶点, 在计算顶点的度之和时每条边都被计算了两次, 所以所有顶点的度之和偶数。无向连通图对应的生成树也是无向连通图, 但此时边数等于顶点数减 1, II 错误。考虑 2 个或以上的顶点恰好构成一个环的情况, 此时每个顶点的度都是 2, III 错误。

17. C

题干要求无论如何分配边, 都能使 7 个顶点连通, 这不同于只要 6 条边两两相连就能构成一个连通图的情形。考虑最极端的情形, 即图 G 的某 6 个顶点构成一个完全无向图, 此时若再添加一条边, 则都将连通第 7 个顶点, 使该图变成一个连通图。所以最少边数 $= 6 \times 5 / 2 + 1 = 16$ 。若边数 n 小于或等于 15, 可以使这 n 条边仅连接图 G 中的某 6 个顶点, 从而导致第 7 个顶点无法与这 6 个顶点构成连通图 (不满足“在任何情况下”)。

为简单起见, 以 5 个顶点为例, 左边 4 个顶点和 $4 \times 3 / 2 = 6$ 条边构成一个完全图, 此时若再添加一条边 (可以是虚线中的任意一条), 则能保证这 5 个顶点在任何情况下都是连通的, 如下图所示。若边数小于 7, 则不能保证 5 个顶点在任何情况下都是连通的。



18. B

无向图边数的 2 倍等于各顶点度数的总和。要求至少的顶点数, 应使每个顶点的度取最大, 而由于其他顶点的度均小于 3, 因此可设它们的度都为 2, 并设它们的数量是 x , 列出方程 $4 \times 3 + 3 \times 4 + 2x = 16 \times 2$, 解得 $x=4$ 。因此至少包含 $4+4+3=11$ 个顶点。

19. D

对于此类分析图的边数、顶点数与连通性问题, 思路是寻找临界情况, 在临界情况下任意增加或减少一条边, 都会改变图的连通性。第一种临界情况如图 1 所示, 此时若减少任意一条边, 图就由连通变为不连通, 即无向图连通的最小边数是 $|V|-1$, 因此, 当 $|E| < |V|-1$ 时, 图一定不连通, C 错误, D 正确。第二种临界情况如图 2 所示, 此时若增加任意一条边, 则图就由不连通变为连通, 即无向图不连通的最大边数是 $(|V|-1)(|V|-2)/2$ (此时 $|V|-1$ 个顶点构成一个完全图), 因此, 只有当 $|E| \geq (|V|-1)(|V|-2)/2 + 1$ 时, 才能保证无向图一定连通, A、B 错误。

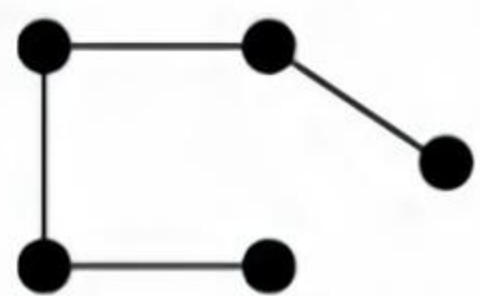


图 1

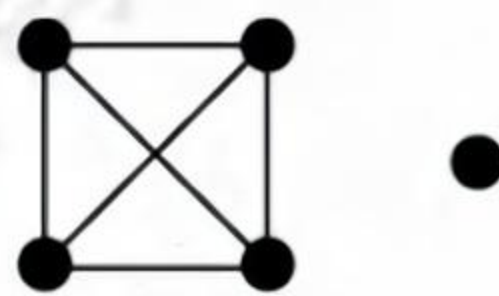


图 2

6.2.7 答案与解析

一、单项选择题

01. B

n 个顶点的图，若采用邻接矩阵表示，不考虑压缩存储，则存储空间大小为 $O(n^2)$ ，A 正确。邻接表可用于存储无向图，只是把每条边都视为两条方向相反的有向边，因此需要存储两次，B 错误。因为邻接矩阵中对角线以下的元素全为 0，所以若存在 $\langle i, j \rangle$ ，则必有 $i < j$ ，由传递性可知图中路径的顶点编号是依次递增的，假设存在环 $k \rightarrow \dots \rightarrow j \rightarrow k$ ，由题设可知 $k < j < k$ ，矛盾，所以不存在环，拓扑序列必定存在，C 正确。D 显然正确。

注 意

若邻接矩阵对角线以下（或以上）的元素全为 0，则图中必然不存在环，即拓扑序列一定存在，但这并不能说明拓扑序列是唯一的。

02. C

除主对角线上的元素外，其余元素全为 1，说明任意两个顶点之间都有边相连，因此该图一定是完全图。

03. D

在无向图的邻接矩阵中，矩阵大小为 n^2 ，非零元素的个数为 $2e$ ，所以零元素的个数为 $n^2 - 2e$ 。读者应掌握此题的变体，即当无向图变为有向图时，能够求出零的个数和非零的个数。

04. D

带权有向图的邻接矩阵中，0 和 ∞ 表示的都不是有向边，而入度是由邻接矩阵的列中元素计算出来的；出度是由邻接矩阵的行中元素计算出来的。

05. B、D

有向图的入度是其第 i 列的非 0 元素之和, 无向图的度是第 i 行或第 i 列的非 0 元素之和。

06. B、B、D

邻接矩阵的顶点数等于矩阵的行(列)数, 有向图的边数等于矩阵中非零元素的个数, 无向图的边数等于矩阵中非零元素个数的一半。

注 意

本题中所给的矩阵为对称矩阵, 若不是对称矩阵, 则必然不可能是无向图。

07. B

邻接矩阵表示唯一是因为图中边的信息在矩阵中有确定的位置, 邻接表不唯一是因为邻接表的建立取决于读入边的顺序和边表中的插入算法。

08. A

设图 G 的邻接矩阵为 A , A^n 的元素 a_{ij}^n 等于从顶点 i 到 j 的长度为 n 的路径的数目, 因此 $a_{ij}^2 = 3$ 表示从顶点 i 到 j 存在 3 条长度为 2 的路径。该结论记住即可。

09. A

邻接表存储时, 顶点数 n 决定了顶点表的大小, 边数 e 决定了边表结点的个数, 且无向图的每条边存储两次, 总存储空间为 $O(n+2e)$ 。而邻接矩阵只与图的顶点数有关, 为 $O(n^2)$ 。

10. D

无向图采用邻接表表示时, 每条边存储两次, 所以其边表结点的个数为偶数。题中边表结点为奇数个, 所以必然是有向图, 且有奇数条边。

11. C

题中的边表是不包括顶点表的。因为任何顶点 u 对应的边表中存放的都是以 u 为起点的边所对应的另一个顶点 v 。从而 v 在边表中出现的次数也就是它的入度。

12. B

最多有 $n(n-1)/2$ 条边, 每条边在邻接表中存储两次, 因此边表结点最多为 $n(n-1)$ 个。

13. A

建立图的邻接表需要遍历所有的顶点和边, 每个顶点有一个顶点表结点, 每条边需要创建一个边表结点并插入到相应的链表中。因此, 共需 $n+2e$ 次操作, 时间复杂度为 $O(n+e)$ 。

14. C

与顶点 v 相关的边包括出边和入边, 对于出边, 只需遍历 v 的顶点表结点和其指向的边表; 对于入边, 则需遍历整个边表。先删除出边: 删除 v 的顶点表结点的单链表, 出边数最多为 $n-1$, 时间复杂度为 $O(n)$; 再删除入边: 扫描整个边表(即扫描剩余全部顶点表结点及其指向的边表), 删除所有的顶点 v 的入边, 时间复杂度为 $O(n+e)$ 。所以总时间复杂度为 $O(n+e)$ 。

15. D

无向图的邻接表中, 第 i 个顶点的度为第 i 个链表中的结点数, 所以选项 A 错。邻接表和邻接矩阵对于不同的操作各有优势, 选项 B 和 C 都不准确。有向图结点的度包括出度和入度, 对于出度, 需要遍历顶点表结点所对应的边表; 对于入度, 则需要遍历剩下的全部边表。

16. A

邻接多重表是无向图的存储结构。

17. B

十字链表是有向图的存储结构。

18. C

邻接矩阵 A 为非对称矩阵, 说明图是有向图, 度为入度与出度之和。各顶点的度是矩阵中此结点对应的行 (对应出度) 和列 (对应入度) 的非零元素之和。

6.3.5 答案与解析

一、单项选择题

01. A

广度优先搜索以起始结点为中心，一层一层地向外层扩展遍历图的顶点，因此无法考虑到边权值，只适合求边权值相等的图的单源最短路径。广度优先搜索相当于树的层序遍历，选项 III 错误。广度优先搜索需要用到队列，深度优先搜索需要用到栈，选项 IV 正确。

02. D

图的深度优先遍历序列通常是不唯一的，I 错误。图 1 是一个不存在回路的有向图，从顶点 1 开始执行广度优先遍历，若不设置访问标志位，则会重复访问顶点 3，II 错误。深度优先遍历（见本节后面习题的解析）或拓扑排序算法可以判断有向图中是否有环，III 正确。图 2 是一个非强连通图，但从顶点 1 开始调用一次广度优先遍历算法就可访问所有顶点，IV 错误。

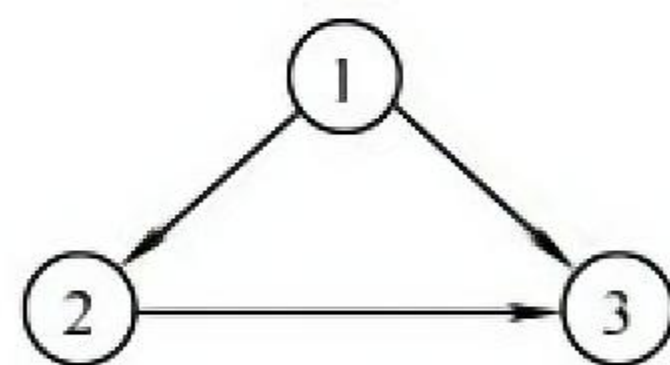


图 1

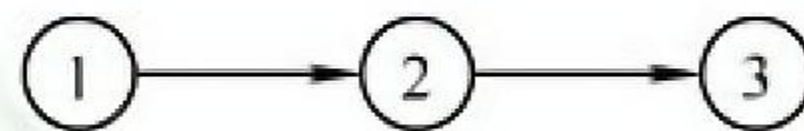


图 2

03. C

DFS（或 BFS）可用来计算无向图的连通分量数，因为一次遍历必然会将一个连通图中的所

有顶点都访问到, 所以计算图的连通分量数正好是 `DFS_Traverse()` 中 DFS 被调用的次数。

04. C、A、C、A

深度优先遍历时, 每个顶点表结点和每个边表结点均查找一次, 每个顶点递归调用一次, 需要借助一个递归工作栈; 而广度优先遍历时, 也是每个顶点表结点和每个边表结点均查找一次, 需要借助一个辅助队列。因此, 时间复杂度都是 $O(n+e)$, 空间复杂度都是 $O(n)$ 。

05. A

在图的广度优先遍历算法中, 每个顶点被访问后立即做访问标记并入队。若队列不空, 则队首顶点出队, 若该顶点的邻接顶点未被访问, 则访问之, 做访问标记并入队; 若被访问过, 则跳过, 如此反复, 直至队空。因此, 在广度优先遍历过程中, 每个顶点最多入队一次。

06. A、A

采用邻接矩阵表示时, 查找一个顶点所有出边的时间复杂度为 $O(n)$, 共有 n 个顶点, 所以时间复杂度均为 $O(n^2)$ 。

07. D

画出草图后, 此类题可以根据边的邻接关系快速排除错误选项。以 A 为例, 在遍历到 e 之后, 应该访问与 e 邻接但未被访问的结点, (e, c) 显然不在边集中。

08. D

仅 1 和 4 正确。以 2 为例, 遍历到 c 之后, 与 c 邻接且未被访问的结点为空集, 所以应为 a 的邻接点 b 或 e 入栈。以 3 为例, 因为遍历要按栈退回, 所以是先 b 后 c , 而不能先 c 后 b 。

09. B、D

图的深度优先搜索类似于树的先根遍历, 即先访问结点, 再递归向外层结点遍历, 都采用回溯算法。图的广度优先搜索类似于树的层序遍历, 即一层一层向外层扩展遍历, 都需要采用队列来辅助算法的实现。

10. A、B

DFS 序列产生的路径为 $\langle 1, 2 \rangle, \langle 2, 5 \rangle, \langle 5, 4 \rangle, \langle 3, 6 \rangle$; BFS 序列产生的路径为 $\langle 1, 2 \rangle, \langle 1, 4 \rangle, \langle 2, 5 \rangle, \langle 3, 6 \rangle$ 。

11. D

画出 V 和 E 对应的图 G , 然后根据搜索算法求解。

这里应注意: 为什么本题序列是不唯一的, 而上题序列却是唯一的呢?

因为上题给出了具体的存储结构, 此时就必须按照算法的过程来执行, 每个顶点的邻接点的顺序已固定, 但本题中每个顶点的邻接点的顺序是非固定的。

12. C

利用深度优先遍历可以判断图 G 中是否存在回路。

对于无向图来说, 若深度优先遍历过程中遇到了回边, 则必定存在环; 对于有向图来说, 这条回边可能是指向深度优先森林中另一棵生成树上的顶点的弧; 但是, 从有向图的某个顶点 v 出发进行深度优先遍历时, 若在 $\text{DFS}(v)$ 结束之前出现一条从顶点 u 到顶点 v 的回边, 且 u 在生成树上 v 的子孙, 则有向图必定存在包含顶点 v 和顶点 u 的环。

13. B

连通分量是无向图的极大连通子图, 其中极大的含义是将依附于连通分量中顶点的所有边都加上, 所以连通分量中可能存在回路, 这样就不是生成树了。

注 意

极大连通子图是无向图（不一定连通）的连通分量，极小连通子图是连通无向图的生成树。极小和极大是在满足连通的前提下，针对边的数目而言的。极大连通子图包含连通分量的全部边；极小连通子图（生成树）包含连通图的全部顶点，且使其连通的边数最少。

14. A

对于无向图的广度优先搜索生成树，起点到其他顶点的路径是图中对应的最短路径，即所有生成树中树高最小。此外，深度优先总是尽可能“深”地搜索图，因此其路径也尽可能长，所以深度优先生成树的树高总是大于或等于广度优先生成树的树高。

15. C

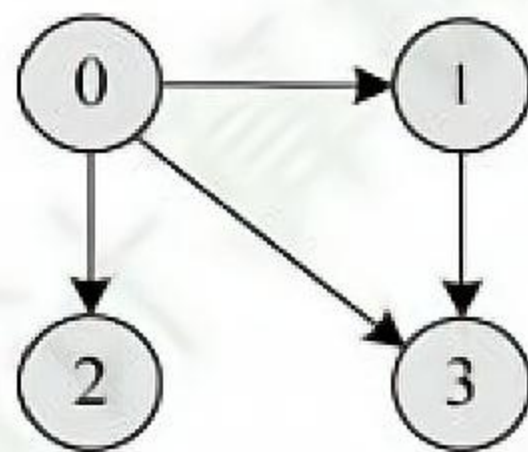
广度优先遍历需要借助队列实现。采用邻接表存储方式对图进行广度优先遍历时，每个顶点均需入队一次（顶点表遍历），所以时间复杂度为 $O(n)$ ，在搜索所有顶点的邻接点的过程中，每条边至少访问一次（出边表遍历），所以时间复杂度为 $O(e)$ ，算法总的时间复杂度为 $O(n+e)$ 。

16. D

只要掌握 DFS 和 BFS 的遍历过程，便能轻易解决。逐个代入，手工模拟，选项 D 是深度优先遍历，而不是广度优先遍历。

17. D

画出该有向图，如下图所示。采用图的深度优先遍历，共有 5 种可能： $\langle v_0, v_1, v_3, v_2 \rangle$ ， $\langle v_0, v_2, v_3, v_1 \rangle$ ， $\langle v_0, v_2, v_1, v_3 \rangle$ ， $\langle v_0, v_3, v_2, v_1 \rangle$ ， $\langle v_0, v_3, v_1, v_2 \rangle$ 。

**18. D**

按深度优先遍历的策略进行遍历。对于 A：先访问 V_1 ，然后访问与 V_1 邻接且未被访问的任意一个顶点（满足的有 V_2 、 V_3 和 V_5 ），此时访问 V_5 ，然后从 V_5 出发，访问与 V_5 邻接且未被访问的任意一个顶点（满足的只有 V_4 ），然后从 V_4 出发，访问与 V_4 邻接且未被访问的任意一个顶点（满足的只有 V_3 ），然后从 V_3 出发，访问与 V_3 邻接且未被访问的任意一个顶点（满足的只有 V_2 ），结束遍历。B 和 C 的分析方法与 A 相同。对于 D，首先访问 V_1 ，然后从 V_1 出发，访问与 V_1 邻接且未被访问的任意一个顶点（满足的有 V_2 、 V_3 和 V_5 ），然后从 V_2 出发，访问与 V_2 邻接且未被访问的任意一个顶点（满足的只有 V_5 ），按规则本应该访问 V_5 ，但 D 却访问了 V_3 ，错误。

6.4.7 答案与解析

一、单项选择题

01. A

当无向连通图存在权值相同的多条边时，最小生成树可能是不唯一的；另外，由于这是一个无向连通图，因此最小生成树必定存在，从而选 A。

02. C

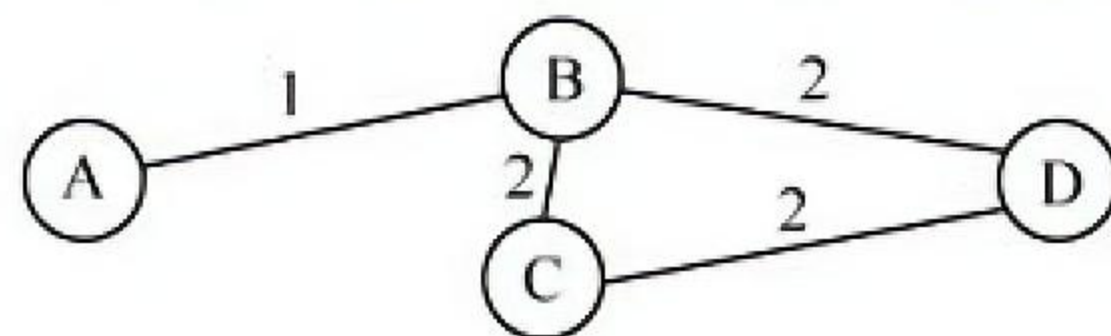
因为无向连通图的最小生成树不一定唯一，所以用不同算法生成的最小生成树可能不同，但当无向连通图的最小生成树唯一时，不同算法生成的最小生成树必定是相同的。

03. A

最小生成树算法是基于贪心策略的，每次总是选取权值最小且满足条件的边，若各边权值不同，则每次选择的新顶点也是唯一的，因此最小生成树唯一，A 正确。对于 B，若无向图本身就是一棵树，则最小生成树就是它本身，这时就是唯一的。对于 C，选取的 $n-1$ 条边可能构成回路。对于 D，含有 n 个顶点、 $n-1$ 条边的子图可能构成回路，也可能不连通。

04. A

若图的边数小于 $n-1$ ，则图不存在最小生成树；若无向连通图的边数等于 $n-1$ ，则最小生成树唯一，即为图本身，所以图的边数一定大于 $n-1$ ，A 正确。若最小生成树不唯一，则一定存在权值相等的边，但未必是权值最小的边，如下图所示，B 错误。最小生成树可能不唯一，但代价一定相同，C 错误。当图的各边的权值互不相等时，图的最小生成树是唯一的，D 错误。



05. A

$U = \{1, 2, 3\}$, $V - U = \{4, 5, \dots\}$, 候选边只能是这两个顶点集之间的边，只有 A 符合题意。

06. C

若选取边(1, 3)则会构成回路。

07. A

A 正确，见严蔚敏撰写的教材《数据结构》。Dijkstra 算法适合求解有回路的带权图的最短路径，也可以求任意两个顶点的最短路径，不适合求带负权值的最短路径问题。在用 Floyd 算法求两个顶点的最短路径时，当最短路径发生更改时， $path_{k-1}$ 就不是 $path_k$ 的子集。

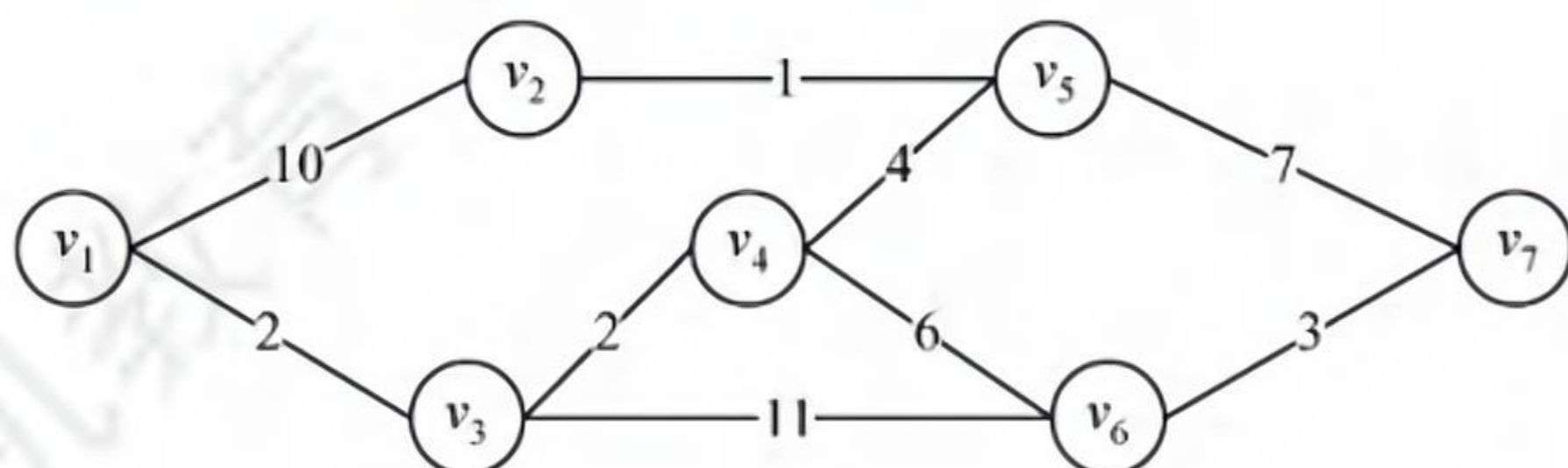
08. C

在负权图中，Dijkstra 算法既不能保证每次选出的顶点都是真正的最近顶点，又不能保证已

确定的最短路径不再被改变, 因此 Dijkstra 算法不允许边的权为负, I 正确。求每对顶点间的最短路径需要调用 Dijkstra 算法 n 次, 时间复杂度为 $O(n^3)$, II 错误。Floyd 算法求每对顶点间的最短路径允许有负边存在, 但不允许有包含负边组成的回路, III 正确。

09. B

题目所描述的图 G 如下图所示。A, B, C, D 对应的路径长度分别为 18, 13, 15, 24。应用 Dijkstra 算法不难求出最短路径为 $v_1 \rightarrow v_3 \rightarrow v_4 \rightarrow v_6 \rightarrow v_7$ 。



10. D

在 Dijkstra 算法的执行过程中, 只可能修改从源点 0 到集合 $V-S$ 中某个顶点的最短路径。

11. A

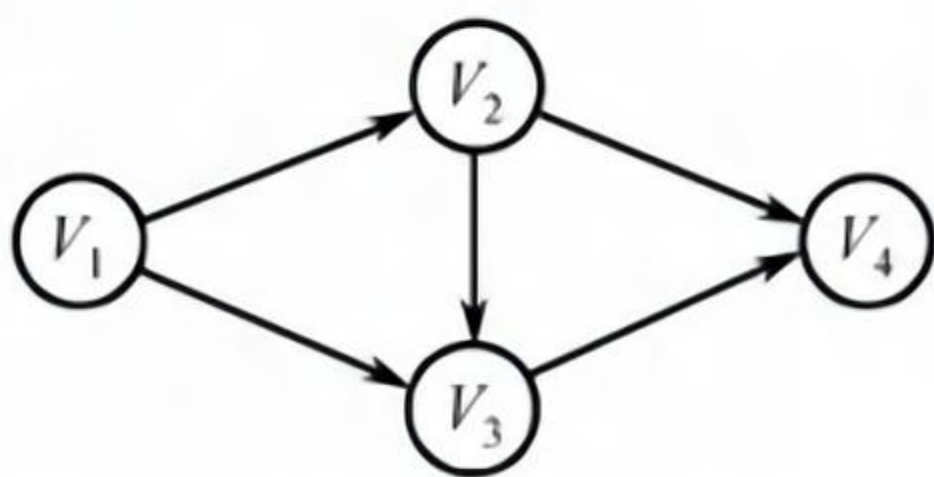
使用深度优先遍历, 若从有向图上的某个顶点 u 出发, 在 DFS(u) 结束之前出现一条从顶点 v 到 u 的边, 由于 v 在生成树上是 u 的子孙, 图中必定存在包含 u 和 v 的环, 因此深度优先遍历可以检测一个有向图是否有环。拓扑排序时, 当某顶点不为任何边的头时才能加入序列, 存在环时环中的顶点一直是某条边的头, 不能加入拓扑序列。也就是说, 还存在无法找到下一个可以加入拓扑序列的顶点, 则说明此图存在回路。求最短路径是允许图有环的。至于关键路径能否判断一个图有环, 则存在一些争议。关键路径本身虽然不允许有环, 但求关键路径的算法本身无法判断是否有环, 判断是否有环是求关键路径的第一步——拓扑排序。

12. D

若图 G 中存在一条从 v_j 到 v_i 的路径, 说明 v_j 是 v_i 的前驱, 则要把 v_j 消去以后才能消去 v_i , 从而拓扑序列中必然先输出 v_j , 再输出 v_i , 这显然与题意矛盾。

13. D

对于 I, 若有向图中存在环, 运行拓扑排序算法后, 肯定会剩下有环的子图, 在此环中无法再找到入度为 0 的顶点, 拓扑排序也就无法再运行。对于 II, 若两个顶点之间不存在祖先或子孙关系, 则它们在拓扑序列中的前后关系是任意的, 因此使用栈和队列都可以, 因为进栈或队列的都是入度为 0 的顶点。III 是难点, 若拓扑序列唯一, 则很自然联想到一个线性的有向图, 下图的拓扑序列也唯一, 但却不满足该条件。对于 IV, 若入度为 0 的顶点不唯一, 则这些顶点均可作为拓扑序列的起点; 若出度为 0 的顶点不唯一, 则这些顶点均可作为拓扑序列的终点。



14. A

强连通图是指有向图中任意顶点对之间都存在两条相反的路径, 这意味着强连通图中一定存在环, 因此不能进行拓扑排序, I 正确。假设顶点 a 和 b 的入度均为 0, 且分别有两条弧从 a 和 b 指向同一顶点 c , 则产生的拓扑序列可以是 abc , 但是此时并无一条弧 $\langle a, b \rangle$, II 错误。

15. D

一个有向图中的顶点不能排成一个拓扑序列, 表明其中存在一个顶点数目大于 1 的回路

(环), 该回路构成一个强连通分量, 从而答案选 D。

16. C

本图的拓扑排序序列有 $ABCFDEG$, $ABCD FEG$, $ABCDEF G$, $ABDC FEG$ 和 $ABDCEFG$ 。读者应能把这一类经典习题的拓扑序列全部写出来。

17. A

拓扑排序的过程: 找到入度为 0 的顶点, 删除该顶点及其所有出边, 并将顶点加入拓扑序列, 重复直至所有顶点都加入拓扑序列。选择入度为 0 的顶点 v_1 , 删除与 v_1 有关的边; 此时顶点 v_3 的入度为 0, 选择 v_3 , 删除与 v_3 有关的边; 以此类推, 得出 G 的拓扑序列。

18. B

无向图的邻接矩阵存储中, 每条边存储两次, 且 $A[i][j] = A[j][i]$ 。

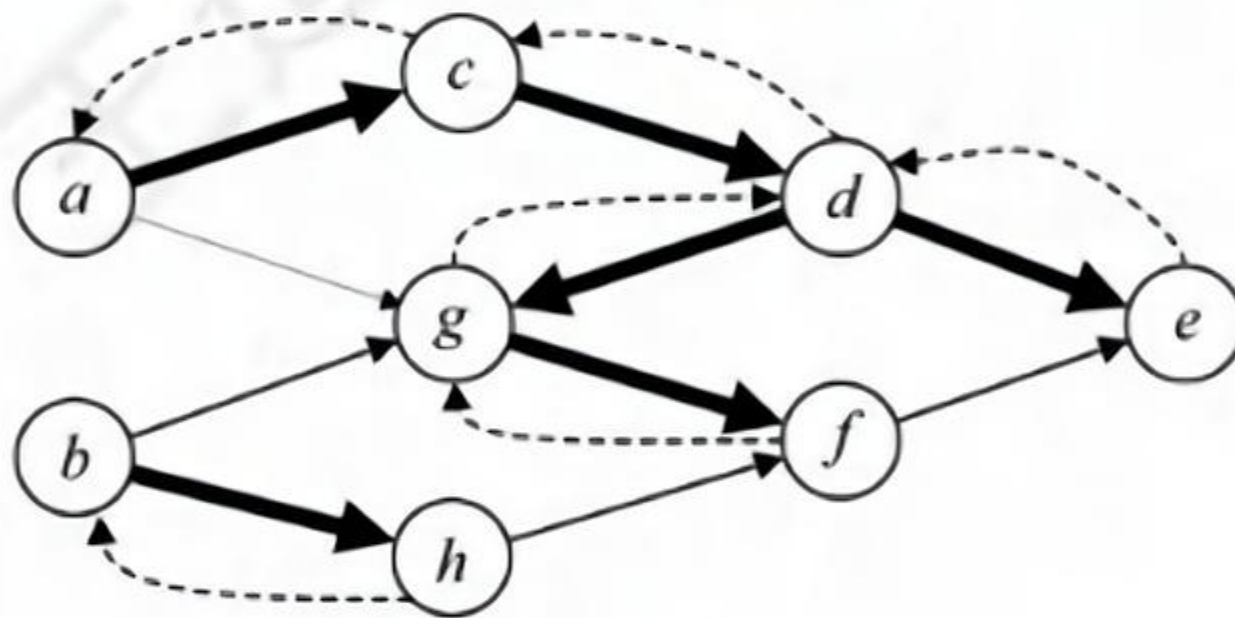
19. C

此题一直以来争议较大, 因为有些书中漏掉了“有序”二字。可以证明, 对有向图中的顶点适当地编号, 使其邻接矩阵为三角矩阵且主对角元素全为零的充分必要条件是, 该有向图可以进行拓扑排序。若这个题目把“有序”二字去掉, 显然应选 D。但此题题干中已经指出是“有序的拓扑序列”, 因此应选 C。需要注意的是, 若一个有向图的邻接矩阵为三角矩阵 (对角线以上或以下的元素为 0), 则图中必不存在环, 因此其拓扑序列必然存在。

20. A

设图中有顶点 v_i , 它有后继顶点 v_j , 即存在边 $\langle v_i, v_j \rangle$ 。根据 DFS 的规则, v_i 入栈后, 必先遍历完其后继顶点后 v_i 才会出栈, 也就是说 v_i 会在 v_j 之后出栈, 在如题所指的过程中, v_i 在 v_j 后打印。由于 v_i 和 v_j 具有任意性, 因此由上面的规律看出, 输出顶点的序列是逆拓扑有序序列。

对有向无环图利用深度优先搜索进行拓扑排序的例子如下: 如下图所示, 退出 DFS 栈的顺序为 $efgdcahb$, 此图的一个拓扑序列为 $bhacd gfe$ 。该方法的每一步均是先输出当前无后继的结点, 即对每个结点 v , 先递归地求出 v 的每个后继的拓扑序列。

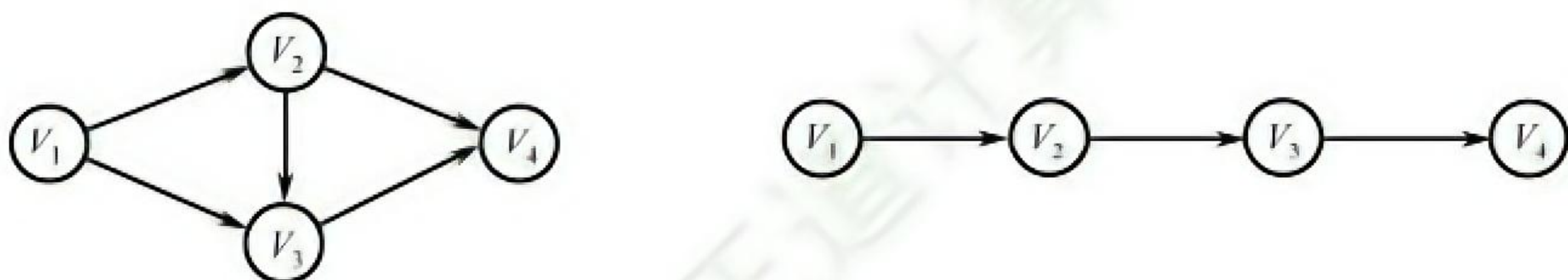


21. C

有向图邻接矩阵的第 V 行中 1 的个数是顶点 V 的出度, 而有向图中顶点的度为入度与出度之和, I 错。无向图的邻接矩阵一定是对称矩阵, 但当有向图中任意两个顶点之间有边相连, 且是两条方向相反的有向边时, 有向图的邻接矩阵也是一个对称矩阵, II 错。最小生成树中的 $n-1$ 条边不能保证是图中权值最小的 $n-1$ 条边, 因为权值最小的 $n-1$ 条边并不一定能使图连通。在下图中, 左图的最小生成树如下图所示, 权值为 3 的边不在其最小生成树中。



有向无环图的拓扑序列唯一并不能唯一确定该图。在下图所示的两个有向无环图中, 拓扑序列都为 V_1, V_2, V_3, V_4 , IV 错。注意, 很多辅导书对该命题的判断是错误的。

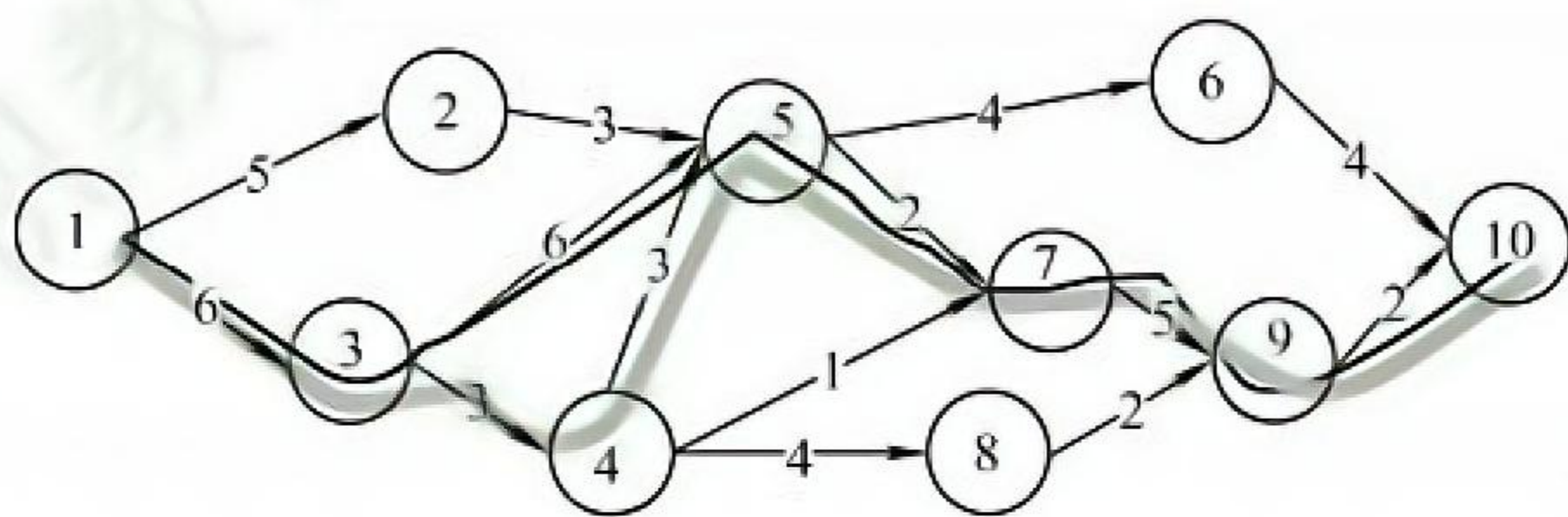


22. C

观察题图，从 V_0 到 V_8 的最长路径为 $V_0 \rightarrow V_1 \rightarrow V_4 \rightarrow V_6 \rightarrow V_8$ ，长度为 $6 + 1 + 9 + 2 = 18$ 。

23. C

题目描述的图如下，得到关键路径的长度为 21，图中画出的两条路径都是关键路径。



24. C

一个事件的最迟发生时间 = $\min\{\text{以该事件为尾的弧的活动的最迟开始时间}\}$ ，或 $\min\{\text{以该事件为尾的弧所指事件的最迟发生时间与该弧的活动的持续时间之差}\}$ 。改变 AOE 网中任何一个活动的持续时间，需要重新计算关键活动，可能导致关键路径的改变。

25. C

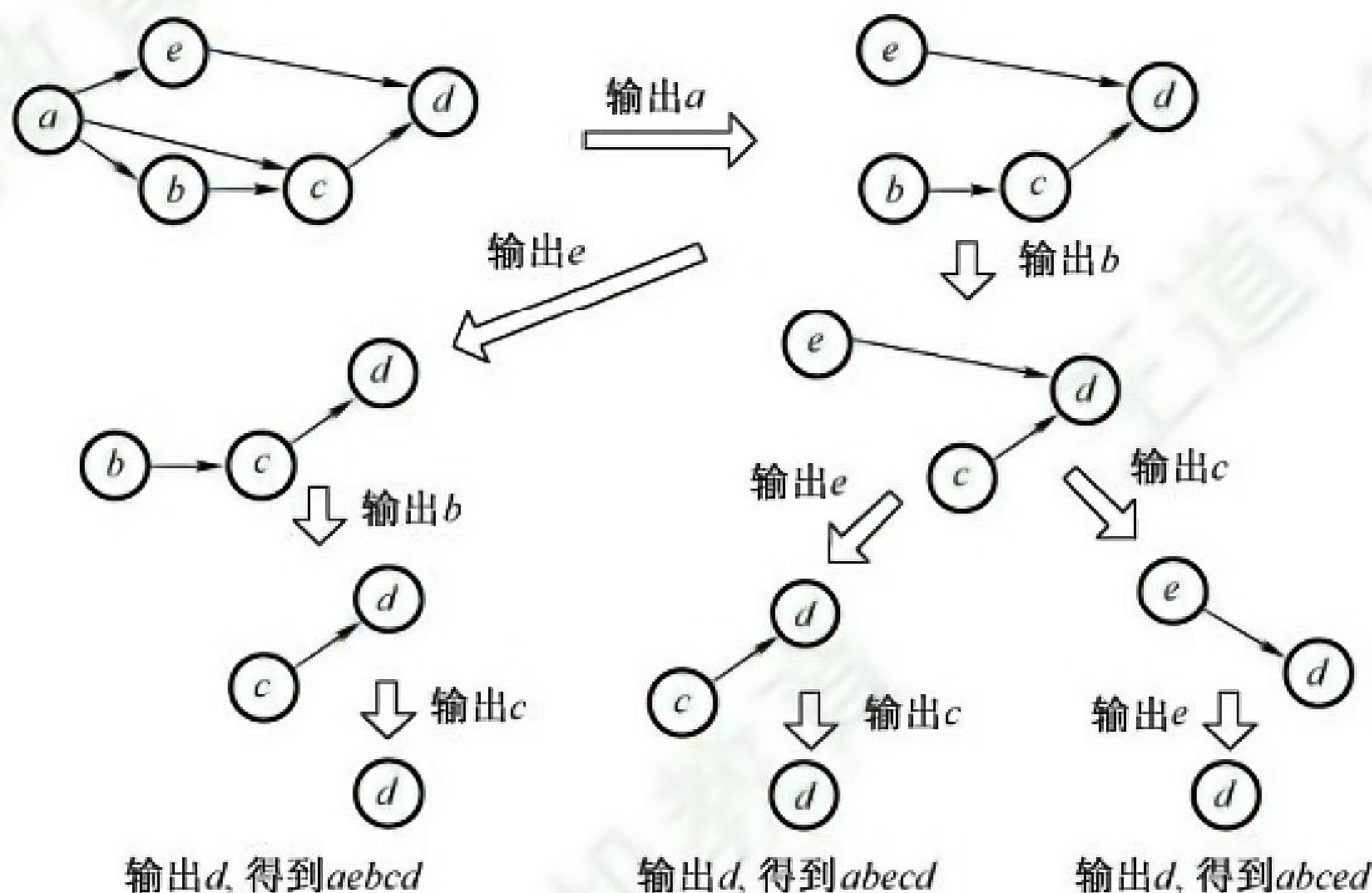
若改变的是所有关键路径上的公共活动，则不一定会产生不同的关键路径（延长必然不会导致，只有缩短才有可能导致）。根据关键路径的定义，可知选项 II 正确。关键路径是源点到终点的最长路径，只有所有关键路径的长度都缩短时，整个图的关键路径才能有效缩短，但也不能任意缩短，一旦缩短到一定程度，该关键活动就可能变成非关键活动。

26. A

邻接矩阵第 i 列值全为 ∞ ，说明顶点 i 没有入边，为整个工程的开始，若关键路径存在，则该顶点一定是起点。不能确定关键路径是否存在，也不能确定其对应的无向图的连通分量个数。

27. B

拓扑排序的过程如下图所示。



可以得到 3 种不同的拓扑序列，即 $abced$ 、 $abecd$ 和 $aebcd$ 。

28. A

最小生成树的树形可能不唯一（因为可能存在权值相同的边），但代价一定是唯一的，选项 I 正确。若权值最小的边有多条并且构成环状，则总有权值最小的边将不出现在某棵最小生成树中，选项 II 错误。设 N 个结点构成环， $N-1$ 条边权值相等，另一条边权值较小，则从不同的顶点开始 Prim 算法会得到 $N-1$ 种不同的最小生成树，选项 III 错误。当最小生成树唯一时（各边的权值不同），Prim 算法和 Kruskal 算法得到的最小生成树相同，选项 IV 错误。

29. C

从 a 到各顶点的最短路径的求解过程下：

顶点	第 1 轮	第 2 轮	第 3 轮	第 4 轮	第 5 轮
b	$(a, b) 2$				
c	$(a, c) 5$	$(a, b, c) 3$			
d	∞	$(a, b, d) 5$	$(a, b, d) 5$	$(a, b, d) 5$	
e	∞	∞	$(a, b, c, e) 7$	$(a, b, c, e) 7$	$(a, b, d, e) 6$
f	∞	∞	$(a, b, c, f) 4$		
集合 S	$\{a, b\}$	$\{a, b, c\}$	$\{a, b, c, f\}$	$\{a, b, c, f, d\}$	$\{a, b, c, f, d, e\}$

后续目标顶点依次为 f, d, e 。

本题也可用排除法：对于 A，若下一个顶点为 d ，路径 a, b, d 的长度为 5，而 a, b, c, f 的长度仅为 4，显然错误。同理可排除选项 B。将 f 加入集合 S 后，采用上述方法也可排除选项 D。

30. C

对角线以下元素均为零，表明只有顶点 i 到 j ($i < j$) 可能有边，而顶点 j 到 i 一定没有边，即有向图是一个无环图，因此一定存在拓扑序列。对于拓扑序列是否唯一，试举一例：设有向图的邻接矩阵

$\begin{bmatrix} 0 & 1 & 1 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$ ，存在两个拓扑序列，因此该图存在可能不唯一的拓扑序列。

结论：对于任一有向图，若它的邻接矩阵中对角线以下（或以上）的元素均为零，则存在拓扑序列（可能不唯一）。反正，若图存在拓扑序列，却不一定能满足邻接矩阵中主对角线以下的元素均为零，但是可以通过适当地调整结点编号，使其邻接矩阵满足前述性质。

31. C

找出 AOE 网的全部关键路径为 $bdcg$ 、 $bdeh$ 和 bfn 。根据性质，只有当所有关键路径的活动时间同时减少时，才能缩短工期。即正确选项中的路径必须能涵盖所有的关键路径。选项 A 和 B 不能涵盖 bfn 这条路径，选项 C 不能涵盖 $bdcg$ 和 $bdeh$ 这两条路径，只有选项 D 能涵盖所有关键路径，因此只有加快 f 和 d 的进度才能缩短工期（建议在图中检验）。

32. D

按照拓扑排序的算法，每次都选择入度为 0 的结点从图中删除，此图中一开始只有结点 3 的入度为 0；删除结点 3 后，只有结点 1 的入度为 0；删除结点 1 后，只有结点 4 的入度为 0；删除结点 4 后，结点 2 和结点 6 的入度都为 0，此时选择删除不同的结点，会得出不同的拓扑序列，分别处理完毕后可知可能的拓扑序列为 3, 1, 4, 2, 6, 5 和 3, 1, 4, 6, 2, 5，选 D。

33. C

从 V_4 开始，Kruskal 算法选中的第一条边一定是权值最小的 (V_1, V_4) ，选项 B 错误。由于 V_1 和 V_4 已经可达，因此含有 V_1 和 V_4 的权值为 8 的第二条边一定符合 Prim 算法，排除 A、D。

34. C

第一个顶点和最后一个顶点相同的路径称为回路；序列中顶点不重复出现的路径称为简单路径；回路显然不是简单路径，I 错误。稀疏图是边比较少的情况，邻接矩阵存储的空间复杂度为 $O(n^2)$ ，必将浪费大量的空间，而邻接表存储的空间复杂度为 $O(n+e)$ ，所以应选用邻接表，II 错误。存在回路的有向图不存在拓扑序列，若拓扑排序输出结束后所余下的顶点都有前驱，则说明只得到了部分顶点的拓扑有序序列，图中存在回路，III 正确。

35. B

根据 Dijkstra 算法，从顶点 1 到其余各顶点的最短路径如下表所示。

顶 点	第 1 轮	第 2 轮	第 3 轮	第 4 轮	第 5 轮
2	5 $v_1 \rightarrow v_2$	5 $v_1 \rightarrow v_2$			
3	∞	∞	7 $v_1 \rightarrow v_2 \rightarrow v_3$		
4	∞	11 $v_1 \rightarrow v_5 \rightarrow v_4$	11 $v_1 \rightarrow v_5 \rightarrow v_4$	11 $v_1 \rightarrow v_5 \rightarrow v_4$	11 $v_1 \rightarrow v_5 \rightarrow v_4$
5	4 $v_1 \rightarrow v_5$				
6	∞	9 $v_1 \rightarrow v_5 \rightarrow v_6$	9 $v_1 \rightarrow v_5 \rightarrow v_6$	9 $v_1 \rightarrow v_5 \rightarrow v_6$	
集合 S	{1, 5}	{1, 5, 2}	{1, 5, 2, 3}	{1, 5, 2, 3, 6}	{1, 5, 2, 3, 6, 4}

快速解法。依次观察从顶点 1 到其他顶点的最短路径长度：顶点 1 到顶点 2 的最短路径长度为 5；顶点 1 到顶点 3 的最短路径长度为 $5 + 2 = 7$ ；顶点 1 到顶点 4 的最短路径长度为 11；顶点 1 到顶点 5 的最短路径长度为 4；顶点 1 到顶点 6 的最短路径长度为 $4 + 5 = 9$ ；最终 $\text{dist}[] = \{0, 5, 7, 11, 4, 9\}$ ，根据 dist 数组值从小到大选择顶点顺序为 1, 5, 2, 3, 6, 4。

36. B

采用邻接表作为 AOV 网的存储结构进行拓扑排序，需要对 n 个顶点做进栈、出栈、输出各一次，当处理 e 条边时，需要检测这 n 个顶点的边链表结点，共需要的时间为 $O(n+e)$ 。若采用邻接矩阵作为 AOV 网的存储结构进行拓扑排序，在处理 e 条边时需对每个顶点检测相应矩阵中的某一行，寻找与它相关联的边，以便对这些边的入度减 1，需要的时间代价为 $O(n^2)$ 。

【补充】有两种常用的拓扑排序算法：基于 BFS 的算法和基于 DFS 的算法。本题未指明采用哪种算法，因此只需验证一种算法即可（说明两种算法在对应条件下的时间复杂度相同）。

基于 BFS 的算法的思想：首先找到所有入度为 0 的结点，将它们加入一个队列，并将它们作为拓扑序列的起始部分；然后依次从队列中取出结点，并删除它们与后继结点的所有边。若某个后继结点的入度变为 0，则将它也加入队列，并将它加入拓扑序列，重复这个过程。

基于 DFS 的算法的思想：在 DFS 调用过程中设定一个时间标记，当 DFS 调用结束时，对各结点计时，进而按结束时间从大到小排序，可以得到一个拓扑序列。

37. D

拓扑排序每次选取入度为 0 的结点输出，经观察不难发现拓扑序列前两位一定是 1, 5 或 5, 1（因为只有 1 和 5 的入度均为 0，且其他结点都不满足仅有 1 或仅有 5 作为前驱）。

38. C

活动 d 的最早开始时间等于该活动弧的起点所表示的事件的最早发生时间，活动 d 的最早开始时间等于事件 2 的最早发生时间 $\max\{a, b+c\} = \max\{3, 12\} = 12$ 。活动 d 的最迟开始时间等于该活动弧的终点所表示的事件的最迟发生时间与该活动所需时间之差，先算出图中关键路径长

度为 27 (对于不复杂的选择题, 找出所有路径计算长度), 那么事件 4 的最迟发生时间为 $\min\{27 - g\} = \min\{27 - 6\} = 21$, 活动 d 的最迟开始时间为 $21 - d = 21 - 7 = 14$ 。

常规方法: 按照关键路径算法算得到下表。

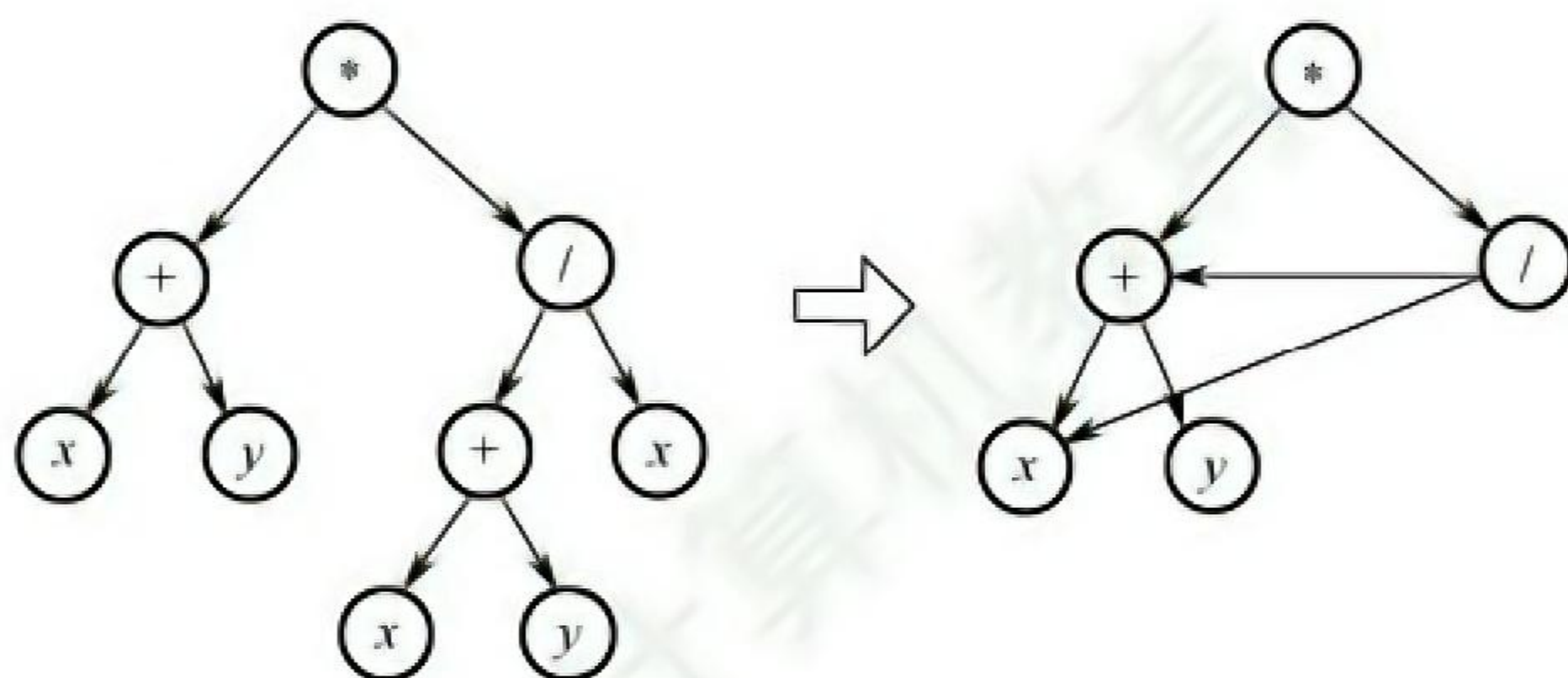
	v_1	v_2	v_3	v_4	v_5	v_6
$v_e(i)$	0	12	8	19	18	27
$v_l(i)$	0	12	8	21	18	27

	a	b	c	d	e	f	g	h
$e(i)$	0	8	0	12	12	8	19	18
$l(i)$	9	8	0	14	12	8	21	18
$l(i) - e(i)$	9	0	0	2	0	0	2	0

从表中可知, 活动 d 的最早开始时间和最迟开始时间分别为 12 和 14, 所以选 C。

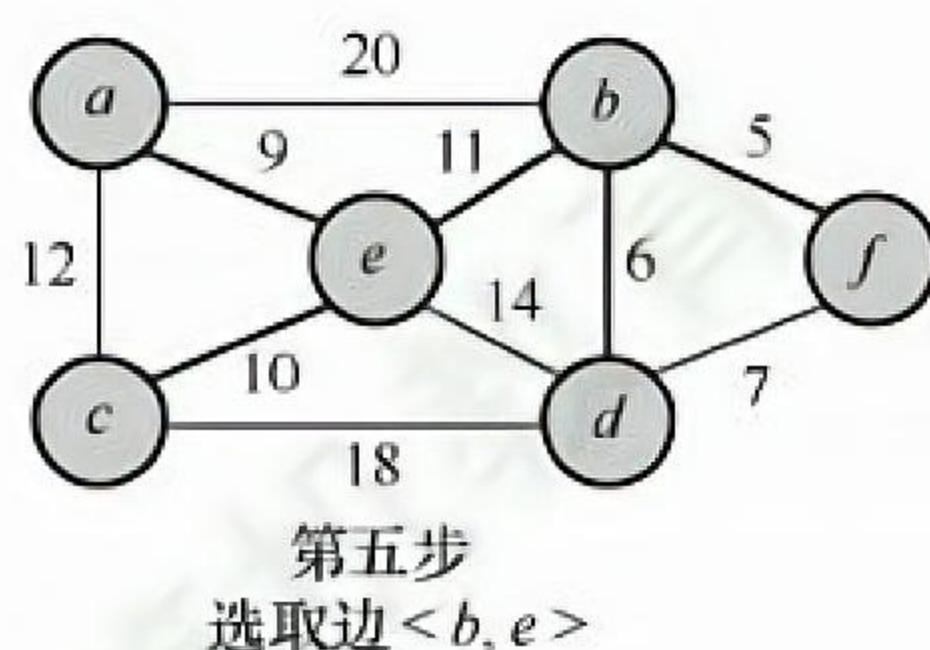
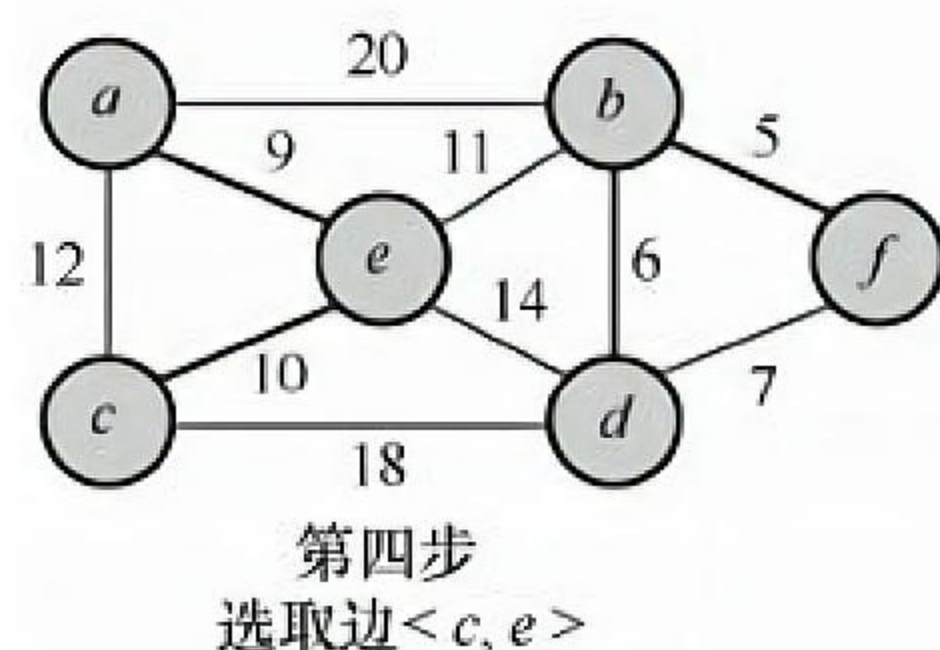
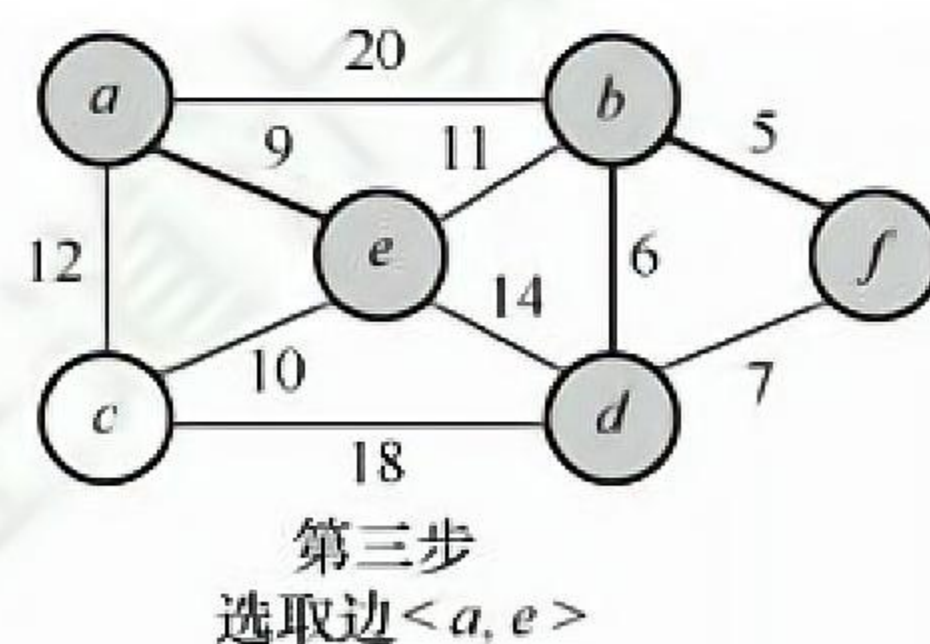
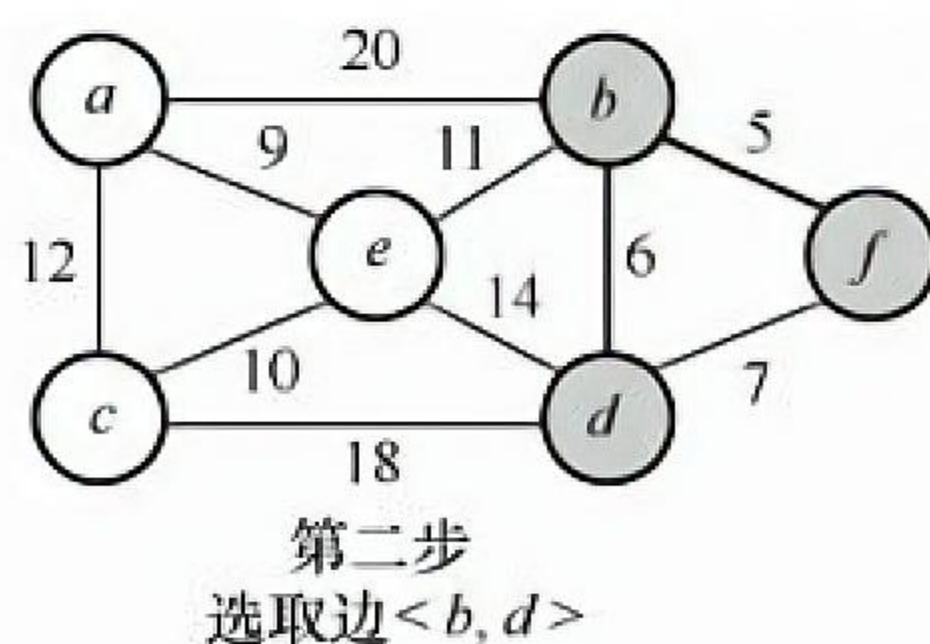
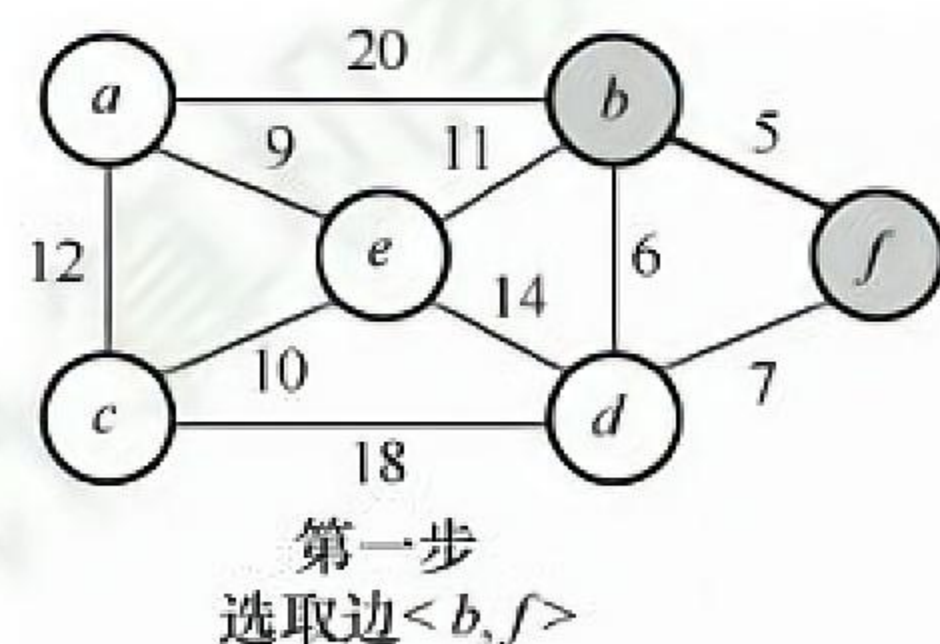
39. A

先将该表达式转换成有向二叉树, 该二叉树中有些顶点是重复的, 为了节省存储空间, 去除重复的顶点, 将有向二叉树去重转换成有向无环图, 如下图所示。



40. A

Kruskal 算法: 按权值递增顺序依次选取 $n-1$ 条边, 并保证这 $n-1$ 条边不构成回路。初始构造一个仅含 n 个顶点的森林; 第一步, 选取权值最小的边 (b, f) 加入最小生成树; 第二步, 剩余边中权值最小的边为 (b, d) , 加入最小生成树, 第二步操作后权值最小的边 (d, f) 不能选, 因为会与之前已选取的边形成回路; 接下来依次选取权值 9、10、11 对应的边加入最小生成树, 此时 6 个顶点形成了一棵树, 最小生成树构造完成。按照上述过程, 加到最小生成树的边依次为 (b, f) , (b, d) , (a, e) , (c, e) , (b, e) 。其生成过程如下所示。



41. B

根据题干所提供的信息可知:

- ①图 G 为有向无环图, 因此一定存在拓扑序列和逆拓扑序列。
 - ②DFS 的性质是顶点 v_i 的所有后继顶点 v_j 出栈后, v_i 才会出栈。
 - ③本题要求执行输出语句后立刻退出递归, 即执行完输出语句后立即出栈, 因此后进栈的顶点先输出, 结合②不难得出只有输出顶点 v_i 的所有后继顶点 v_j 后, v_i 才会输出。
- 综合上述分析, 输出的顶点序列是逆拓扑有序序列。

42. B

关键路径是指权值之和最大而非边数最多的路径, A 错误。选项 B 是关键路径的概念。无论是存在一条还是存在多条关键路径, 增加任意一个关键活动的时间都会延长工程的工期, 因为关键路径始终是权值之和最大的那条路径, C 错误。仅有一条关键路径时, 减少关键活动的时间会缩短工程的工期; 存在多条关键路径时, 缩短一条关键活动的时间不一定会缩短工程的工期, 缩短了路径长度的那条关键路径不一定还是关键路径, D 错误。

43. A

求拓扑序列的过程: 从图中选择无入边的结点, 输出该结点并删除该结点的所有出边, 重复上述过程, 直至全部结点都已输出, 这样求得的拓扑序列为 ABCDEF。每次输出一个结点并删除该结点的所有出边后, 都发现有且仅有一个结点无入边, 因此该拓扑序列唯一。

44. C

在执行 Dijkstra 算法时, 首先初始化 $\text{dist}[]$, 若顶点 1 到顶点 i ($i = 2, 3, 4, 5$) 有边, 就初始化为边的权值; 若无边, 就初始化为 ∞ ; 初始化顶点集 S 只含顶点 1。Dijkstra 算法每次选择一个到顶点 1 距离最近的顶点 j 加入顶点集 S , 并判断由顶点 1 绕行顶点 j 后到任意一个顶点 k 是否距离更短, 若距离更短 (即 $\text{dist}[j] + \text{arcs}[j][k] < \text{dist}[k]$), 则将 $\text{dist}[x]$ 更新为 $\text{dist}[j] + \text{arcs}[j][k]$; 重复该过程, 直至所有顶点都加入顶点集 S 。数组 dist 的变化过程如下图所示, 可知将第二个顶点 5 加入顶点集 S 后, 数组 dist 更新为 21, 3, 14, 6。

$$\text{dist}\{26, 3, \infty, 6\} \xrightarrow{\text{顶点3入S}} \{25, 3, \infty, 6\} \xrightarrow{\text{顶点5入S}} \{21, 3, 14, 6\}$$

45. B

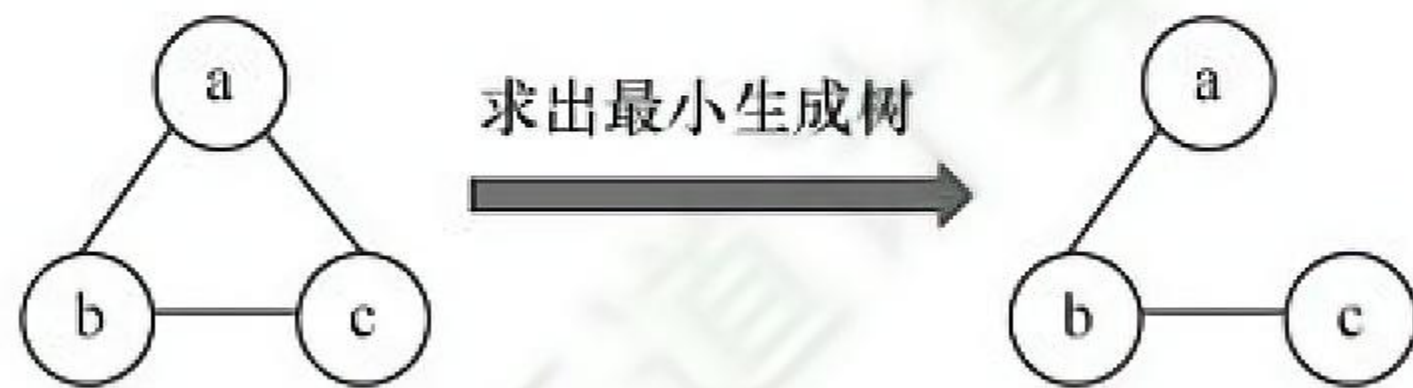
在 AOE 网中, 活动的时间余量 = 结束顶点的最迟开始时间 - 开始顶点的最早开始时间 - 该活动的持续时间。根据关键路径算法得到下表:

结点编号	1	2	3	4	5	6
最早开始时间 $v_e(i)$	0	2	5	8	9	12
最迟开始时间 $v_l(i)$	0	4	5	8	11	12

c 的时间余量 = $v_l(3) - v_e(2) - 1 = 5 - 2 - 1 = 2$, g 的时间余量 = $v_l(6) - v_e(3) - 1 = 12 - 5 - 1 = 6$, h 的时间余量 = $v_l(5) - v_e(4) - 1 = 11 - 8 - 1 = 2$, j 的时间余量 = $v_l(6) - v_e(5) - 1 = 12 - 9 - 1 = 2$ 。

46. B

Prim 算法和 Kruskal 算法用于求解最小生成树, 最小生成树中某顶点到其余各顶点的路径不一定具有最短路径的性质。例如, 在下图所求得的最小生成树中, a 到 c 的路径长度为 2, 但原图中 a 到 c 的最短路径长度为 1。



图的广度优先搜索算法总按距离由近到远来遍历图中的每个顶点，因此可用来求解非带权图（或各边权值均相同）的单源最短路径问题。